Cracking the Crackme

Sexy 1337

Md. Moniruzzaman Prodhan
(NomanProdhan)

31 October 2024

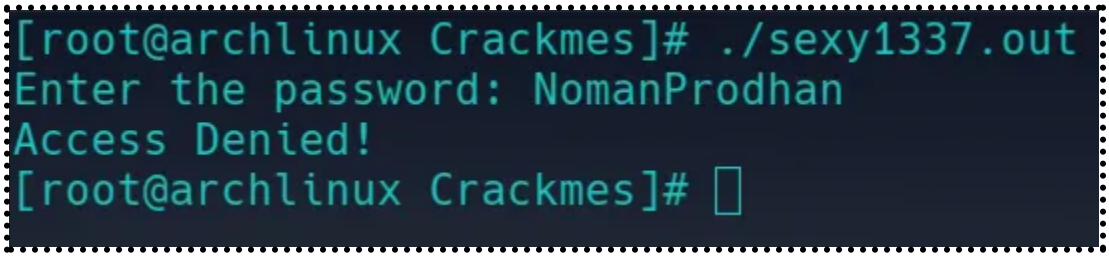
Hey Cracker!

I'm back with another write-up, and this time, I'll be diving into cracking the "Sexy 1337" binary from crackmes.one! This crackme was created by RafaelGFreveng. You can download the crackme from here.

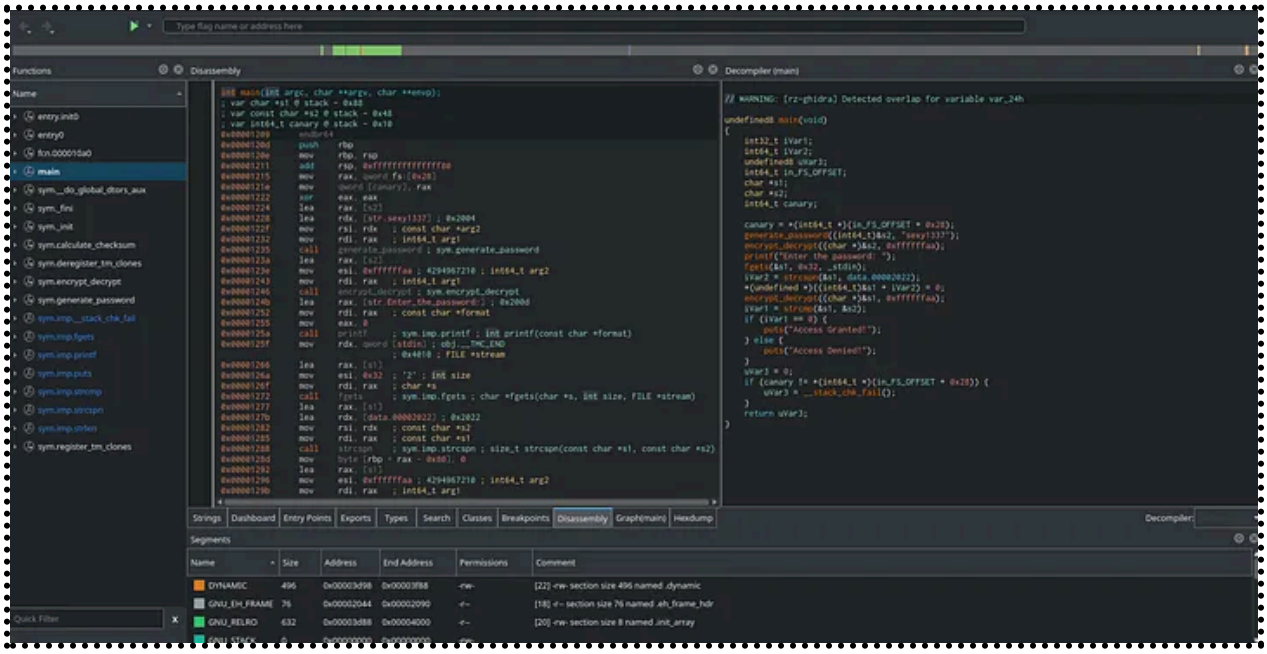
Let's dive into the analysis and see how to crack this challenge!

First up, I ran the binary. It started by displaying a prompt asking for password, something like this:

I entered my name as password and hit enter. The binary responded with another message, like this:



Now, it's time to get into the real analysis. I'll load the binary into Cutter to analyze it and find the right password.



We can see that the binary is written in C and, as expected, starts execution main function. This is where I'll start my analysis. After decompiling the main function, I get the following code:

```
undefined8 main(void)
{
    int32_t iVar1;
    int64_t iVar2;
    undefined8 uVar3;
    int64_t in_FS_OFFSET;
    char *s1;
    char *s2;
    int64_t canary;

    canary = *(int64_t *)(in_FS_OFFSET + 0x28);
    generate_password((int64_t)&s2, "sexy1337");
    encrypt_decrypt((char *)&s2, 0xffffffaa);
    printf("Enter the password: ");
    fgets(&s1, 0x32, _stdin);
    iVar2 = strcspn(&s1, data.00002022);
    *(undefined *)((int64_t)&s1 + iVar2) = 0;
    encrypt_decrypt((char *)&s1, 0xffffffaa);
    iVar1 = strcmp(&s1, &s2);
    if (iVar1 == 0) {
        puts("Access Granted!");
    } else {
```

```
        puts("Access Denied!");
    }
    uVar3 = 0;
    if (canary != *(int64_t *)(in_FS_OFFSET + 0x28)) {
        uVar3 = __stack_chk_fail();
    }
    return uVar3;
}
```

In the main function, several variables are defined, including a canary, which we can safely ignore for now. However, two functions immediately caught my attention and those are generate_password and encrypt_decrypt.

When I decompiled generate_password I got the following code.

```
void generate_password(int64_t arg1, char *arg2)
{
    uint64_t uVar1;
    char *s;
    int64_t var_20h;
    int64_t var_18h;
    size_t var_10h;

    uVar1 = strlen(arg2);
    for (var_18h = 0; (uint64_t)var_18h < uVar1; var_18h = var_18h + 1) {
        *(char *)(var_18h + arg1) = arg2[var_18h] + '\x03';
    }
    *(undefined *)(uVar1 + arg1) = 0;
    return;
}
```

It appears that the function adds '\x03' to each character of arg2, which is the string "**sexy1337**".

In the main function, encrypt_decrypt is called immediately after generate_password.

So, I decompiled the encrypt_decrypt function.

```
void encrypt_decrypt(char *arg1, int64_t arg2)
{
    char *var_20h;
```

```
    int64_t var_ch;

    for (var_ch._0_4_ = 0; arg1[(int32_t)var_ch] != '\0'; var_ch._0_4_ =
(int32_t)var_ch + 1) {
        arg1[(int32_t)var_ch] = arg1[(int32_t)var_ch] ^ (uint8_t)arg2;
    }
    return;
}
```

The encrypt_decrypt function takes two parameters: the output from generate_password, stored in s2, and the constant 0xffffffaa. After generate_password creates a modified version of "sexy1337" by adding \x03 to each character, encrypt_decrypt further processes this result by performing an XOR operation on each character of s2 using 0xffffffaa.

After performing the operations in the main function with generate_password and encrypt_decrypt, the program then takes our input as a potential password. It runs our input through encrypt_decrypt with the same key (0xffffffaa), effectively encrypting it in the same way as the original password.

Finally, the program compares our encrypted input with the previously generated and encrypted password. An if condition checks if both encrypted values match. If they are the same, the program displays "Access Granted!" If they differ, it shows "Access Denied!"

Since we understand how generate_password generate the correct password, we can use a Python script to reproduce that algorithm and generate the correct password. Here's a script that mimics the functionality of generate_password:

```
knownString = "sexy1337"
correctPass = ""
for char in knownString:
   correctPass += chr(ord(char) + 3)
print("Generated password:", correctPass)
```

I hope you enjoyed the writeup, I will be back with another new writeup. Until then, happy cracking.🤟🤟